TP5 – MPI: Point-to-Point & Collectives

Code Analysis & Performance Report

Kawtar Labzae

February 2025

Contents

1 Exercise 1 – Hello World **2**
1.1 Code and Key Points . . . 2
1.2 Answer to Question 4 . . . 2

2 Exercise 2 – Sharing Data **2**
2.1 Code and Key Points . . . 2

3 Exercise 3 – Ring Broadcast **3**
3.1 Code and Key Points . . . 3

4 Exercise 4 – Parallel Matrix-Vector Product **4**
4.1 Code and Key Points . . . 4
4.2 Performance Results ($N = 10\,000$, 10 processes) . . . 4

5 Exercise 5 – Parallel π Calculation **5**
5.1 Mathematical Background . . . 5
5.2 Code and Key Points . . . 5
5.3 Performance Results ($N = 10^9$) . . . 6

6 Summary **7**

1 Exercise 1 – Hello World

1.1 Code and Key Points

Listing 1: ex1.c – Basic MPI skeleton

```c
#include <mpi.h>
#include <stdio.h>

int main(int argc, char *argv[]) {
    int rank, size;
    MPI_Init(&argc, &argv);           // Initialize MPI runtime
    MPI_Comm_size(MPI_COMM_WORLD, &size); // Total number of processes
    MPI_Comm_rank(MPI_COMM_WORLD, &rank); // This process's ID
    printf("I am process %d among %d\n", rank, size);
    MPI_Finalize();                   // Clean up MPI resources
    return 0;
}
```

If we wanted only rank 0 to run we would add a condition on the execution if (rank == 0) and write the execution clause so that only process with rank 0 would run. **Key MPI calls:**

- `MPI_Init` – must be the *first* MPI call; initialises the communication environment.
- `MPI_Comm_size` / `MPI_Comm_rank` – query the global communicator to obtain the number of processes and this process's rank.
- `MPI_Finalize` – must be the *last* MPI call; releases all MPI resources. Omitting it causes undefined behaviour: the program may hang, produce resource leaks, or generate runtime errors depending on the MPI implementation.

1.2 Answer to Question 4

Omitting `MPI_Finalize` leaves MPI in an uninitialised state at exit. Most implementations will print a warning and attempt a forced cleanup, but pending communications may be lost and buffered output may not be flushed. It is a programming error that can cause non-deterministic behaviour in larger runs In my case when I run it, I got the printed sentences but at the end I got a warning and a problem that the process has exited improperly.

2 Exercise 2 – Sharing Data

2.1 Code and Key Points

Listing 2: ex2.c – Broadcast loop until negative integer

```c
#include <mpi.h>
#include <stdio.h>

int main(int argc, char *argv[]) {
    int rank, value;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    value = 0;
    do {
        if (rank == 0) scanf("%d", &value); // Only rank 0 reads input
        MPI_Bcast(&value, 1, MPI_INT, 0, MPI_COMM_WORLD); // Broadcast
            to all
        if (value >= 0)
```

```
            printf("I, process %d, received %d of process 0\n", rank, value);
    } while (value >= 0);
    MPI_Finalize();
}
```

Key points:

- `MPI_Bcast` is a *collective* operation: *all* processes must call it, even those that are not the root. This is why the broadcast is placed *outside* the `if (rank == 0)` block.
- The loop terminates cleanly because rank 0 broadcasts the negative sentinel value, causing all processes to exit the `do-while` simultaneously.
- Only rank 0 calls `scanf`; the other processes block on `MPI_Bcast` until the value is ready.

3 Exercise 3 – Ring Broadcast

3.1 Code and Key Points

Listing 3: ex3.c – Value propagated around a ring of processes

```
if (rank == 0) {
    scanf("%d", &value);
    MPI_Send(&value, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);   // Send to rank 1
    printf("Process 0 sent initial value %d to Process 1\n", value);
    // Close the ring: receive the final accumulated value from the last process
    MPI_Recv(&value, 1, MPI_INT, size-1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    value += rank; // rank 0 adds its own contribution
    printf("Process 0 received final value %d from Process %d\n", value, size-1);
} else {
    MPI_Recv(&value, 1, MPI_INT, rank-1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    value += rank;  // Each process adds its rank
    printf("Process %d received data, added rank. New value: %d\n", rank, value);
    int next = (rank + 1) % size;
    MPI_Send(&value, 1, MPI_INT, next, 0, MPI_COMM_WORLD); // Forward to next
}
```

Key points:

- Uses `MPI_Send` / `MPI_Recv` – the fundamental *point-to-point* (P2P) primitives.
- The modulo `(rank+1) % size` closes the ring: the last process sends back to rank 0.
- If the initial value is v_0, the final value at rank 0 is $v_0 + 0 + 1 + 2 + \ldots + (p-1) = v_0 + \frac{p(p-1)}{2}$ where p is the number of processes.
- **Deadlock risk:** if all processes call `MPI_Send` before `MPI_Recv`, the program can deadlock. The current design avoids this because process 0 sends *before* receiving, and all other processes receive first.

4 Exercise 4 – Parallel Matrix-Vector Product

4.1 Code and Key Points

Listing 4: ex4.c – MPI matrix-vector multiplication with Scatterv/Gatherv

```c
// Broadcast the vector b to all processes
MPI_Bcast(b, size, MPI_DOUBLE, 0, MPI_COMM_WORLD);

// Compute row distribution (handles size % num_procs != 0)
for (int i = 0; i < num_procs; ++i) {
    int local_rows = size / num_procs + (i < (size % num_procs) ? 1 : 0);
    sendcounts[i] = local_rows * size;   // Elements to send to process i
    displs[i]     = offset * size;       // Starting offset in A
    recvcounts[i] = local_rows;
    recv_displs[i]= offset;
    offset += local_rows;
}

// Distribute rows of A
MPI_Scatterv(A, sendcounts, displs, MPI_DOUBLE,
             local_A, my_rows * size, MPI_DOUBLE, 0, MPI_COMM_WORLD);

// Local computation: each process multiplies its rows by b
for (int i = 0; i < my_rows; ++i) {
    local_x[i] = 0.0;
    for (int j = 0; j < size; ++j)
        local_x[i] += local_A[i * size + j] * b[j];
}

// Gather results back to rank 0
MPI_Gatherv(local_x, my_rows, MPI_DOUBLE,
            x_parallel, recvcounts, recv_displs, MPI_DOUBLE, 0,
               MPI_COMM_WORLD);
```

Key points:

- `MPI_Scatterv` / `MPI_Gatherv` (variable-count variants) are used instead of the uniform `MPI_Scatter`/`MPI_Gather` to handle cases where N is not divisible by the number of processes. The extra rows are distributed one-by-one to the first $N \bmod p$ processes.
- `MPI_Bcast` sends the full vector b to every process before computation; this is necessary because each process needs all of b to compute its assigned rows.
- The serial baseline is timed *only on rank 0* before the parallel section, so the speedup comparison is valid.

4.2 Performance Results ($N = 10\,000$, 10 processes)

Metric	Value
Serial Time	1.6597 s
Parallel Time	0.8824 s
Speedup	1.88
Efficiency	18.8%

Analysis: The speedup of only $\approx 1.88\times$ with 10 processes (efficiency 18.8%) is far below the ideal of $10\times$. Several factors explain this:

- **Communication overhead:** `MPI_Scatterv` distributes a $10\,000\times10\,000$ matrix (≈ 800 MB of doubles), which dominates the runtime. The computation itself ($O(N^2/p)$ per process) is dwarfed by data movement.
- **Broadcast of b:** Every process receives the full 10 000-element vector before computing, adding extra communication latency.
- **Amdahl's law:** The matrix initialisation and serial timing block on rank 0 is serial overhead that limits achievable speedup.
- **Memory bandwidth:** Even in the serial case, a $10\,000 \times 10\,000$ double matrix saturates DRAM bandwidth; adding inter-node communication makes things worse.

A better strategy for large matrices would be to keep the matrix distributed and avoid the scatter/gather entirely (as in ScaLAPACK-style approaches).

5 Exercise 5 – Parallel π Calculation

5.1 Mathematical Background

$$\pi \approx \frac{4}{N}\sum_{i=0}^{N-1}\frac{1}{1+x_i^2}, \qquad x_i = \frac{i+0.5}{N}$$

This is the midpoint-rule numerical integration of $\int_0^1 \frac{4}{1+x^2}\,dx = \pi$.

5.2 Code and Key Points

Listing 5: ex5.c – Pi calculation with MPI_Reduce

```c
// Divide iterations among processes (handles N % num_procs != 0)
long long base      = N / num_procs;
long long remainder = N % num_procs;
long long local_N   = base + (rank < remainder ? 1 : 0);
long long start_i   = (rank < remainder)
                      ? rank * (base + 1)
                      : remainder * (base + 1) + (rank - remainder) *
                          base;

MPI_Barrier(MPI_COMM_WORLD); // Synchronise before timing
double start_parallel = MPI_Wtime();

// Each process sums its own slice
double local_sum = 0.0;
for (long long i = start_i; i < start_i + local_N; ++i) {
    double x = (i + 0.5) / (double)N;
    local_sum += 4.0 / (1.0 + x * x);
}

// Sum all local results onto rank 0
double global_sum = 0.0;
MPI_Reduce(&local_sum, &global_sum, 1, MPI_DOUBLE, MPI_SUM, 0,
    MPI_COMM_WORLD);
```

Key points:

- `MPI_Reduce` with `MPI_SUM` replaces an explicit gather + loop; it is both simpler and more efficient.
- `MPI_Barrier` synchronises all processes before the timer starts, preventing any process's initialisation latency from corrupting the measurement.
- The load is perfectly balanced: the $\lfloor N/p \rfloor$ or $\lceil N/p \rceil$ iteration count per process never differs by more than 1.
- The task is *embarrassingly parallel* (no inter-process data dependency during computation), so near-linear speedup is achievable.

5.3 Performance Results ($N = 10^9$)

Processes	Speedup	Efficiency
1	0.935	93.5%
2	1.875	93.8%
4	3.752	93.8%
8	8.109	101.4%
10	10.324	103.2%
16	17.119	107.0%
20	23.505	117.5%

Analysis:

- **Efficiency < 1 at 1 process (93.5%):** The parallel version *includes* a barrier and MPI overhead even with a single process. The serial baseline is computed *first*, when the CPU caches are cold; the "parallel" single-process run benefits from warm caches and branch-predictor state, slightly inverting the comparison.
- **Efficiency > 1 for $p \geq 8$:** This super-linear speedup is a *cache effect*. With $N = 10^9$ iterations, the serial run constantly misses the last-level cache. When the work is split among 8+ processes, each process's working set fits in its local L2/L3 cache, dramatically reducing memory stall cycles. This is not a measurement error – it is a well-known phenomenon in memory-bound loops.
- **Scalability:** The task is embarrassingly parallel with minimal communication (a single `MPI_Reduce` at the end), so the parallel fraction is ≈ 1 and Amdahl's law predicts near-perfect scaling – which the results confirm.
- **Contrast with Exercise 4:** The matrix-vector product is *communication-bound* (large Scatterv), while π calculation is *compute-bound* with negligible communication. This explains why Ex. 4 achieves only 18.8% efficiency while Ex. 5 exceeds 100%.

6 Summary

Exercise	**MPI Primitive(s)**	**Key Observation**
Ex.1 – Hello World	`MPI_Init/Finalize`, `Comm_rank/size`	Foundation skeleton; `MPI_Finalize` is mandatory
Ex.2 – Sharing Data	`MPI_Bcast`	All processes must call collectives; only root reads input
Ex.3 – Ring	`MPI_Send`/`MPI_Recv`	P2P ring; deadlock avoided by careful send/recv ordering
Ex.4 – Matrix-Vector	`MPI_Scatterv`, `MPI_Gatherv`, `MPI_Bcast`	Communication-bound; low efficiency due to data scatter cost
Ex.5 – Pi	`MPI_Reduce`, `MPI_Barrier`	Embarrassingly parallel; superlinear speedup from cache effects